Lab 6: Maps That Don't Lie

Overview

This week, we've used two different approaches to show communciate using spatial data: 1. static choropleth 2. an interactive map a reader can click and drag

But both approaches have the same potential for misrepresentation - the tool will happily produce a *technically correct, substantively misleading* map, and it won't tell you if/when it has.

Sketch 5 already required you build two choropleths of the same column and name which one might mislead a reader. In this lab, you're doing something similar but with a twist: you'll be building maps with the goal of avoiding misrepresentation, and then critiquing a map that was built with the goal of misrepresenting the data.

You have two files you've already used this term: `cuyahoga_tracts_acs.gpkg` (428 census tracts, 8 ACS variables) and `cbw_stations.geojson` (166 Chesapeake tidal monitoring stations), plus `cbw_counties.geojson` for Task B's choropleth layer.

Download the template notebook, `lab06_template.ipynb`, before you start. It has one markdown header and one seeded code (or prose) cell per task below.

Setup: the standard Colab cell

Same cell every Part 2 notebook opens with, plus the mapping libraries:

```
# Run this cell first (takes ~1 minute). Colab already includes geopandas.
%pip install -q libpysal esda splot mapclassify contextily
%pip install -q folium leafmap
```

Data: all three files load by URL:

```
DATA_BASE_P2 = "https://raw.githubusercontent.com/pjbitterman/KSU_x9076/main/data/part2"

# Local path fallback (uncomment if running outside Colab without internet access):
# DATA_BASE_P2 = "../data/part2"
```

- `cuyahoga_tracts_acs.gpkg` — 428 Census-tract polygons, `EPSG:4269`, with eight ACS estimates: `median_hh_income`, `pct_poverty`, `pct_bachelors_plus`, `pct_renter_occupied`, `median_gross_rent`, `median_home_value`, `median_age`, `total_population`. Seven of the eight carry genuine ACS-suppression `NaN`s (a handful to a few dozen tracts each, depending on the column) — `total_population` is the one exception, a full-count Census figure that's never suppressed. Tract `39035990000` is the edge case of open Lake Erie water
- `cbw_stations.geojson` — 166 tidal water-quality monitoring stations (points), with `station_id`, `latitude`, `longitude`, `station_type` (`main`/`trib`), `tidal_status`, `state`, `waterbody`, `cbw_segment`, `min_year`, `max_year`
- `cbw_counties.geojson` — 205 counties/independent cities across the seven-state Chesapeake watershed, with `state_fp`, `county_fp`, `geoid`, `name`, `name_long`, `state`. No numeric attribute of its own

---

Task A: Static choropleth built two ways (22 points)

In class, we compared three classification schemes (quantiles, equal interval, Fisher-Jenks) to demonstrate the scheme determines which tracts look "high" and which look "low," independent of whether the underlying values changed at all. This task asks you to make that choice once, deliberately, and defend it.

1. Load `cuyahoga_tracts_acs.gpkg` into `tracts`.
2. Pick **one** ACS column to map, from: `median_hh_income`, `pct_poverty`, `pct_bachelors_plus`, `pct_renter_occupied`, `median_gross_rent`, `median_home_value`, or `median_age` — any one **except** `total_population`. (`total_population` never has a missing value in this file, so it can't exercise the `missing_kwds` step below — pick one of the other seven.) Report how many tracts are missing your chosen column (`.isna().sum()`).
3. Build **two** choropleths of your chosen column, each with a different classification scheme (`scheme="quantiles"`, `"equal_interval"`, or `"fisher_jenks"` — pick any two of the three). Each map needs a title, a legend with a real title (`legend_kwds`), and a `missing_kwds` entry that visibly marks the tracts missing your column (a solid color, labeled "no

data" or similar — not left to whatever a `NaN` happens to render as by default).

4. For each of your two maps, print the classification object's bin counts — the actual per-bin tract counts, not an eyeballed guess from the map.
5. In 3–5 sentences, using the bin counts you just printed as evidence, state which of your two schemes you'd recommend for showing this variable to a general audience, and why. This should be grounded in something you observed, not a generic "one scheme is usually better" claim.

Choose a `cmap` that's appropriate for your variable

Task B: An interactive map, built for exploring (22 points)

In class, we built a folium map with station markers, popups, and a county choropleth layer. This task requires you to build that same shape of map yourself, on the full 166-station file.

1. Load `cbw_stations.geojson` into `stations` and `cbw_counties.geojson` into `counties`.
2. Build a `folium.Map` (or a `leafmap.Map`, if you prefer that library) centered reasonably on the Chesapeake watershed, and add a marker (or `CircleMarker`) for **every one of the 166 stations**. Each marker's popup must show at least `station_id`, `station_type`, and `waterbody`. 23 of the 166 stations carry a null value in `station_type`/`waterbody`, so your popup text must handle that condition and not crash - and not silently print the literal string `"None"`.
3. Build a per-county station-density number the same way we did in class
4. Add that per-county count as a choropleth layer on your map (`folium.Choropleth(..., key_on="feature.properties.geoid", ...)`, or `leafmap.Map.add_data(counties_counts, column="station_count", scheme="Quantiles", ...)` if using leafmap) with a legend title, plus your station markers from step 2 as a separately toggleable layer (`folium.FeatureGroup` + `folium.LayerControl()`, or leafmap's equivalent layer control).
5. Export the finished map to a standalone `.html` file and download it from Colab's Files panel.

Submit the exported `.html` file alongside your notebook — see Deliverables, below.

Task C: Critique a map that's lying to you (19 points)

The two tasks above asked you to build "honest" maps. The code below produces a map that is misleading. Run it, then write the critique.

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(7, 7))
tracts.plot(
    column="median_hh_income",
    cmap="rainbow",
    vmin=30000,
    vmax=90000,
    legend=True,
    ax=ax,
)
ax.set_title("Cuyahoga County household income by tract")
ax.set_axis_off()
plt.tight_layout()
plt.show()
```

This map has (at least) two deliberate, specific problems, both of which the code above is doing on purpose. Before you write your critique, you may want to explore the results of changing a few of the parameters. It would also be useful to explore the underlying data and compare the data to the map's visual representation.

That's all the hints you get :)

Write approximately 250 words critiquing this map, using the numbers you just computed as your evidence. Your critique must name and explain both problems above your own words, grounded in what you found (not just "the legend is misleading," but *how many tracts* or *why* the method misrepresents them).

Note: since you will be writing code to investigate the map's problems, you should include any code in the template you are submitting.

Deliverables and submission

Submit through Canvas:

1. Your completed template notebook (`lab06_template.ipynb`), containing Task A's two choropleths and bin-count printouts + justification, Task

B's interactive-map-building code, Task C's code and critique. All in one file, runnable top to bottom.

2. Task B's exported `.html` map file.

Point breakdown

Component	Points
Task A: Static choropleth, two ways	22
Task B: Interactive map	22
Task C: Critique a misleading map	19
Style and presentation	12
Total	**75**

"Style and presentation" carries more weight in this lab than any other Part 2 lab so far, since presentation *is* this lab's subject. The component covers: every map having a real title and a legend with a proper title (not the column's raw variable name); sequential color choices appropriate to a one-directional variable; `missing_kwds` (or the equivalent) used wherever a column has `NaN`s; popups that read cleanly and handle nulls honestly; `snake_case` variable names and f-strings throughout; and a notebook that runs top to bottom without manual intervention.

Late work follows the course late policy